MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Single-GPU mixture-of-experts (MoE) inference reads expert weights from HDD when a task's HBM and host allocation cannot retain the model. Initial placement and later GPU evictions determine how often these reads recur. We present MECC, the MoE Expert Cache Compiler, to connect layer-wise initial allocation with joint HBM/CPU retention. The compiler emits capacities and ranked expert IDs. The runtime promotes selected experts and retains GPU victims on the host. Delivery orders reads and copies before consumption. In single-run FP16 Mixtral-8x7B measurements on one A100, MECC with uniform initialization takes 7.38 s per decode token. Decode speedups are 1.77× over streaming and 2.95× over fixed CPU retention. Sample perplexity matches these controls.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models expand parameter capacity while selecting a subset of experts for each token [1]–[4]. Sparse execution reduces arithmetic. Every expert can still be selected by a later token. A single GPU may lack the high-bandwidth memory (HBM) needed to retain that pool. A serving task may also receive less host memory than the server contains. This setting arises when jobs share a host or reserve resources for separate model instances. Expert demand can then reach a hard disk drive (HDD) after missing both fast tiers. Every such miss adds weight delivery to the layer's execution path. The fast tiers avoid different parts of that path. An HBM hit avoids both the cold read and H2D. A host hit avoids the cold read but still needs H2D. A placement decision therefore changes where a later request begins its delivery. A token crosses many MoE layers, so these delays can recur throughout decoding. Retaining weights across tokens can avoid repeated reads without changing the router or precision. We study how to allocate that retention capacity and update its contents during inference. The resource constraint belongs to the serving task: physical server capacity alone does not determine which weights the task can retain. A deployment can trade resident multi-GPU latency for a smaller accelerator allocation.

Expert parallelism distributes the expert pool over several GPUs [5]–[7]. Offloading systems instead manage transfers, host execution, and expert residency [8]–[12]. MoE-CORE combines layer-wise allocation, profile initialization, and online replacement [13]. SSD-LLaMA combines bounded RAM/VRAM caches with storage delivery [14]. Edge FlashMoE studies replacement under device-memory constraints [15]. These systems establish the importance of deciding which experts remain accessible after execution. MECC examines a specific coupling: an offline layer allocation supplies the initial state of a joint HBM/CPU retention rule. Initial coverage determines the first demand reads. The plan and the eviction rule affect different request segments. Loading a frequently selected expert can reduce early reads. Its first GPU eviction creates a later choice: retain it on the host or fetch it again. The host admission can also displace another expert. A plan's initial hit rate does not describe this sequence of choices. Promotion and victim retention determine whether later selections require another cold read. Figure 1 isolates the latter decision in local A100 controls. Independent caches take 32.94 s per decode token, versus 7.38 s for joint retention at the same nominal slots and initialization. Their all-phase HDD-demand shares are 47.9% and 12.5%. The shared loader and initial state connect this difference to the local retention rules.

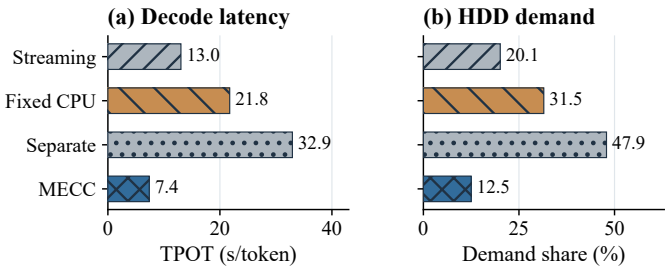


Fig. 1. Local single-A100 cache controls on Mixtral-8x7B. Separate denotes independent HBM/CPU management. All policies use the same nominal slots, initialization, and eight-prompt input. HDD demand spans all instrumented phases; each policy has one run.

We present MECC, the MoE Expert Cache Compiler, for this allocation and retention problem. The compiler takes model metadata, task allowances, and layer profiles. It compares the next uncovered rank in each layer and emits HBM/CPU capacities with ordered expert IDs. Initialization loads those IDs into the fast tiers. During inference, the original router supplies demand. An HBM hit changes recency. A CPU hit promotes its expert and preserves the displaced HBM expert on the host. A cold admission follows the same promotion/demotion rule and can remove a host victim. Capacities remain fixed while resident identities follow the stream. The plan retains layer IDs, so equal expert indices in different layers remain separate objects. An expert can enter HBM even when calibration never selected it. Delivery resolves the selected experts' read and copy dependencies before their GPU consumers execute. The implementation batches cold reads during prefill and uses demand delivery during decode. This separates three responsibilities: selecting an initial state, retaining experts after tier crossings, and making selected weights available for computation. The compiler's output becomes the runtime's starting state; observed requests determine its later membership.

This paper makes three contributions.

(1) **Compiling initial residency:** we specify a layer-wise plan with capacities and expert IDs. A marginal-coverage scan allocates slots under fixed expert allowances.

(2) **Preserving cross-tier reuse:** we connect that plan to a joint promotion/demotion rule. We separate logical membership from the host mirrors and temporary copies used by delivery.

(3) **Characterizing the decisions:** we compare placement, retention, transfer exposure, and workload sensitivity under a shared inference interface. On one A100, uniform-initialized MECC takes 7.38 s/token, versus 13.04 for streaming and 21.75 for fixed CPU retention. These are local policy controls. The evaluation separates initial coverage from later replacement and examines the latency paid for using one accelerator. Comparisons report both response time and accelerator occupancy.

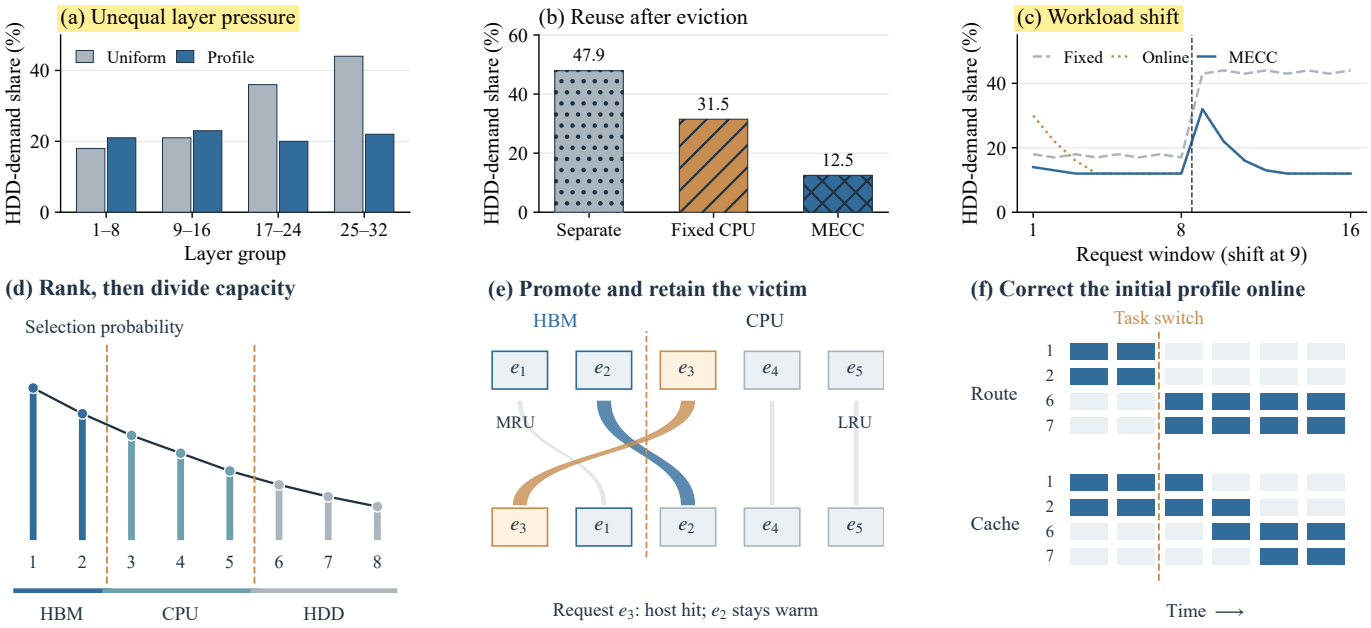


Fig. 2. Motivation evidence and matching insights. (a–c) compare layer pressure, cross-tier reuse, and workload change; (b) counts A100 demand across all phases. (d–f) show layer-specific capacity cuts, joint victim retention, and online membership updates.

II. Background

A. Sparse Execution, Full Weight Capacity

An MoE layer contains a router and a bank of experts. The router selects a small subset for each token, and their weighted outputs form the layer result [1], [2], [16]. Mixtral and the DeepSeek family use this separation between active arithmetic and total model capacity [3], [4], [17]–[19]. Expert identity includes the layer: equal indices in different layers refer to different weights. For input x, let $G(x)$ be the selected set and $g_e(x)$ its normalized gate weights. The layer computes

$$y(x) = \sum_{e \in G(x)} g_e(x) E_e(x). \tag{1}$$

A generated token passes through successive layers, so expert demand is revealed incrementally during the forward pass. Autoregressive decoding revisits these layers with potentially different selections. Residency changes the location of an immutable expert, while its router score and function remain unchanged.

B. Expert Residency on One GPU

With enough aggregate HBM, expert parallelism places experts on different GPUs and moves token activations to their owners [5]–[7], [20], [21]. A single-GPU deployment instead moves selected weights through the memory hierarchy. Dense layers, activations, and the KV cache compete with experts for HBM; KV management is complementary to expert caching [22], [23]. A CPU hit needs a host-to-device (H2D) copy. An HDD miss first reads the expert into host memory. Heterogeneous inference systems expose these capacity and delivery choices [24]–[28]. We use immutable FP16 experts and retain backing weights on HDD. A resident copy avoids a later cold read, making the history of expert reuse relevant even though routing remains sparse. Prefill groups prompt rows by their selected experts. One fetched expert can serve several rows. Decode advances one token per sequence. These phases expose different amounts of work for hiding delivery.

Below full fast-tier coverage, requests can miss both HBM and CPU. MECC addresses this regime by preserving useful copies that avoid another HDD read.

III. Motivation and Insights

A. Allocating Slots by Marginal Demand

A slot can cover different request mass in different layers. Uniform allocation gives each layer the same capacity, even after its hottest experts fit. Figure 2(a,d) connects the resulting pressure to a per-layer capacity cut. The next uncovered expert determines the gain from another slot. Comparing that gain across layers allocates capacity where it covers more demand. Ranking within each layer then determines the initial IDs. Capacity and identity are separate outputs: one sets the room available to a layer, and the other fills it. Routing locality across models has also been studied independently [29].

B. Preserving Reuse after GPU Eviction

A GPU eviction changes an expert's location before it determines the expert's next use. If the host retains the weights, another selection avoids HDD. Figure 2(b,e) shows the promotion/demotion decision. Promotion vacates one CPU position as the HBM victim loses its GPU position. Joint retention reuses that host position for the victim. The local independent-cache control reports 47.9% HDD demand, versus 12.5% for joint retention across all instrumented phases. This comparison concerns the specified policies under the same initial slots. The victim keeps its place in the combined reuse order as it crosses the HBM boundary.

C. Updating Residency beyond Calibration

A profile summarizes calibration inputs. The next prompt can select a different expert set before a fixed initial placement changes. Figure 2(c,f) pairs the demand shift with an update of resident IDs. The compiler supplies capacities and a starting membership. Recency then changes membership within those capacities. Initial coverage controls early reads; online admission controls the response to later selections. These are separate time intervals in one cache lifecycle. A task switch can expose the distinction even when both methods start with the same profile. Updating IDs from actual requests lets the runtime respond without compiling a new plan or predicting a later gate.

IV. Overview

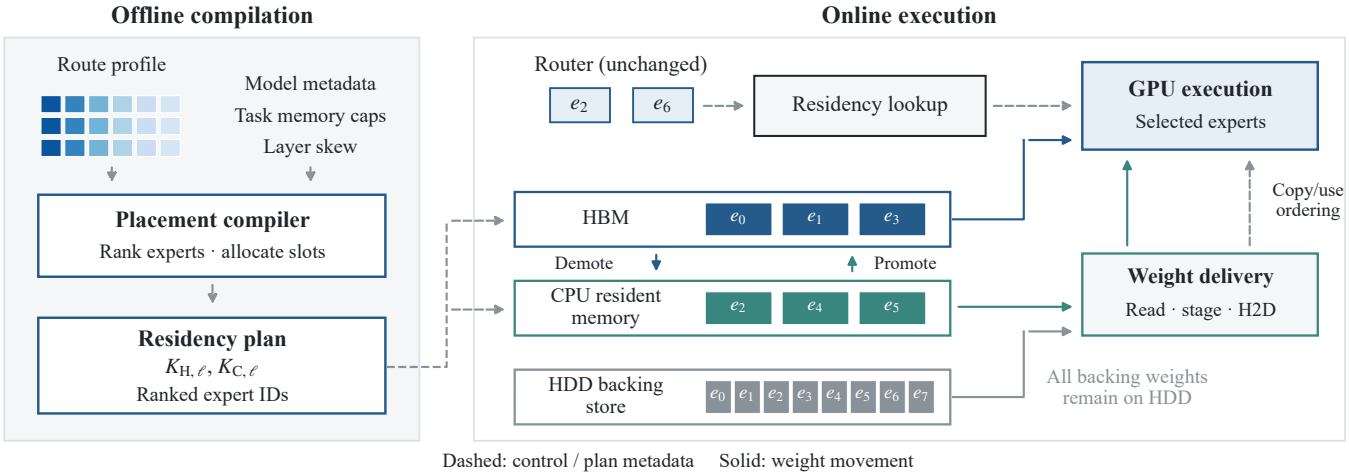


Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution resolves unchanged router selections against HBM, CPU resident memory, and HDD, then delivers weights before their GPU consumers. Dashed arrows carry control or plan metadata; solid arrows move weights.

Figure 3 follows one model from preparation to inference. Offline profiling supplies layer/expert counts and skew parameters. The compiler combines those inputs with model metadata and task allowances. Its plan records per-layer capacities, ordered IDs, and initial tiers. Initialization reads the specified experts from HDD and populates HBM and CPU. The backing store retains the complete pool.

At each MoE layer, the gate produces selected IDs and weights. Residency lookup chooses HBM, CPU, or HDD as the source. Promotion makes the requested expert available on the GPU. Demotion preserves an HBM victim on the host when the retention rule admits it. Read and copy dependencies then order execution before output aggregation. Each generated token repeats this layer sequence; later tokens reuse the resulting resident state.

The interfaces separate what stays fixed from what changes. The plan fixes per-layer capacities. Accesses change membership and recency. Delivery controls when weights can be consumed. Placement comparisons therefore hold the runtime rule fixed. Retention comparisons hold initial capacities and IDs fixed. Scheduling comparisons use the same demand and resident state. This decomposition links the framework to the comparisons in Section VI without treating any one component as a substitute for the others.

V. Method

A. Compiling Capacities and Initial Identities

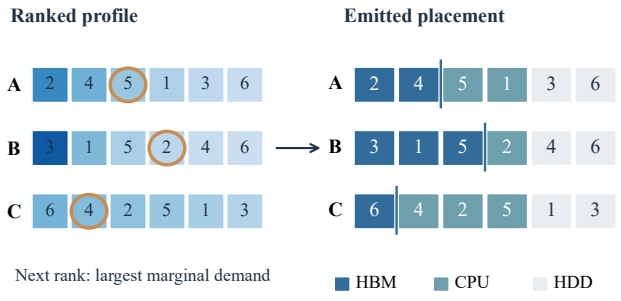


Fig. 4. Compiling the initial state. The left grid contains ranked per-layer demand; circles mark the next uncovered candidates. Shared HBM/CPU slot allowances produce different cuts in the emitted rows. A, B, and C identify different layers. Values and slot counts are schematic.

Consider L MoE layers with N_ℓ experts per layer. The evaluated models use equal-size experts and equal top-k across layers. Let s be expert size and let B_H, B_C be logical expert allowances. They supply $S_H = \lfloor B_H/s \rfloor$ HBM slots and $S_C = \lfloor B_C/s \rfloor$ CPU slots. Dense weights, activations, mirrors, and staging are separate allocations.

The compiler takes an expert order and a skew parameter α_ℓ for each layer. Profile counts determine expert order; a discrete Zipf model supplies the marginal slot score:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{\sum_{j=1}^{N_\ell} j^{-\alpha_\ell}}, \qquad p_{\ell,r} = q_{\ell,r}/L. \tag{2}$$

Here r is a rank within one layer, not an expert ID shared across layers. Equal top-k gives equal layer selection mass. Sorting counts preserves the mapping from each rank to its original expert ID.

Figure 4 shows the resulting cuts. A valid plan obeys $\sum_\ell K_{H,\ell} \le S_H$, $\sum_\ell K_{C,\ell} \le S_C$, and $K_{H,\ell} + K_{C,\ell} \le N_\ell$. The adaptive scan starts with one slot per layer in each fast tier. We consider allowances that support these minima and leave CPU ranks after the HBM cut.

The scan first assigns HBM slots. At each step, it chooses the layer with the largest next-rank score:

$$\Delta_\ell = q_{\ell,K_{H,\ell}+1}. \tag{3}$$

CPU allocation then compares ranks after each HBM cut. Allocating a slot exposes the next rank in that layer. Layer ID breaks score ties; expert ID breaks count ties. A direct scan costs $O(L(S_H + S_C))$ after sorting. It maximizes marginal coverage at each step; it does not solve a joint latency objective.

The plan emits per-layer capacities and ordered IDs. Initialization places the first HBM ranks on the GPU and the next CPU ranks on the host. Missing count data selects natural ID order. Capacities stay fixed during a run. Router requests can replace initial residents, including experts absent from the profile.

B. Retaining Victims across the Tier Boundary

Each layer keeps two ordered resident sets. Most-recently used (MRU) entries precede least-recently used (LRU) entries. HBM holds the first part of the logical order; CPU holds the next part. An HBM hit moves its expert to HBM MRU. A CPU hit removes its host entry and promotes it to HBM MRU. If HBM is full, its LRU victim

becomes CPU MRU. CPU pressure removes the CPU LRU. HDD retains every backing expert.

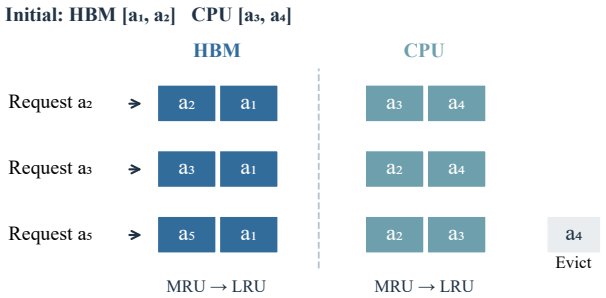


Fig. 5. Three requests from the same initial state. A hit changes recency; a host promotion also demotes the GPU victim; a cold admission can additionally evict the CPU victim. Rows show completed logical states, each ordered MRU to LRU.

Figure 5 starts each row from HBM $[a_1, a_2]$ and CPU $[a_3, a_4]$. Requesting a_2 changes only HBM recency. Requesting a_3 gives HBM $[a_3, a_1]$ and CPU $[a_2, a_4]$. Requesting cold a_5 gives HBM $[a_5, a_1]$ and CPU $[a_2, a_3]$; a_4 leaves the fast tiers. Promotion preserves the more recent GPU victim in the position vacated on the host.

These completed serial transitions implement exclusive LRU across the combined logical capacity. Batch preparation can promote several experts before their consumers run. The serial stack describes replacement, while the batch path also needs temporary copies and a consumption order. Neither this stack rule nor an initial profile guarantees lower latency on every request stream.

The implementation retains a host mirror with each GPU resident. Demotion can reuse that mirror without D2H. Mirrors add physical host bytes; they do not add logical CPU slots. Logical eviction can release a cache entry before CUDA finishes using its storage. Tensor lifetime must therefore be tracked separately from membership.

C. Ordering Delivery before Expert Consumption

Prefill identifies the experts selected across its token rows. The runtime submits cold reads for those experts and resolves each read at consumption. Figure 6 shows a prefill schedule in which a resident expert executes while another read progresses. Decode uses demand delivery rather than this prefill read batch. Both paths consume the original gate output and preserve expert aggregation order.

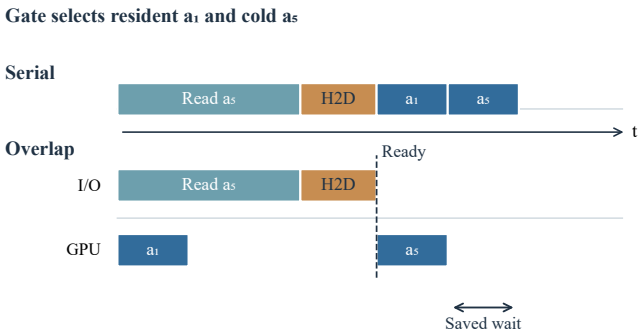


Fig. 6. Prefill delivery of resident a_1 and cold a_5 selected by one gate. Overlap executes ready work during the same read/copy path. The copy/use boundary orders a_5's consumer; the host source remains live through copy completion. Durations are schematic.

An SSD future must complete before its host weights can be copied. The implementation then waits on the copy stream before yielding CUDA weights to a consumer. Recording the consuming stream prevents allocator reuse while kernels still reference the tensors. These operations enforce read, copy, and use dependencies. The current synchronization covers the copy stream; it is not a per-expert readiness-event scheduler.

The schedule can reduce exposed delivery time without changing which requests miss. Its overlap depends on available resident work and on the submitted read batch. Cache coverage and delivery overlap therefore require separate comparisons.

D. Sharing Execution and Accounting

Policies share the FP16 loader and router. Accounting distinguishes unique primary residents, host mirrors, and staging storage. Initialization, warmup, prefill, decode, and PPL use separate windows. Logical slot counts describe the replacement experiment; physical allocations describe the host and GPU resources it consumes.

VI. Evaluation

A. Experimental Setup

Measured runs use FP16 Mixtral-8x7B-Instruct-v0.1 on one A100 80 GB PCIe. The cold store is on a RAID logical device reporting rotational media; physical drives remain uninventoried. Main runs use four HBM and three host slots per layer, uniform initialization, eight WikiText-2 prompts, 64 input and 64 output tokens, and one run per policy. Perplexity uses 1024 WikiText-2 tokens. TTFT is time to first token. TPOT is time per output token during decode. The 50/40 GB planner allowances define logical slots; physical allocations also include mirrors and staging buffers.

Yellow-highlighted values and panel titles denote authored illustrative targets, not measurements. The other numerical results are measured A100 outcomes. The experiments address full-system benefit, individual mechanisms, workload change, resource tradeoffs, and numerical equivalence. Streaming loads selected experts on demand without MECC's retention rules. Fixed CPU reserves a static host set; independent caching updates the two fast tiers separately. RF is a local recency/frequency adapter derived from FlashMoE's public features. It uses HBM LRU and a CPU replacement score. CPU-compute is a separate local adapter. The shared loader keeps model loading and routing fixed across these residency and execution controls.

B. Single-GPU Performance

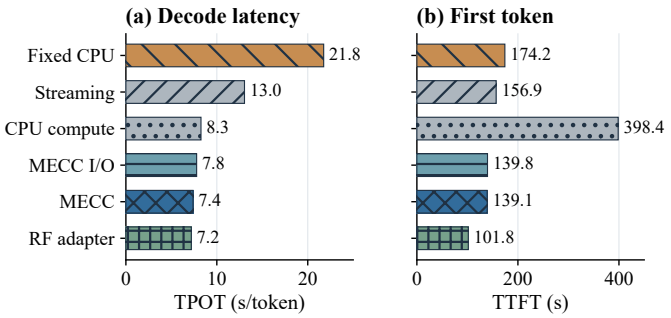


Fig. 7. Measured single-A100 runtime: one run per local policy, eight WikiText-2 prompts, 64 output tokens. RF and CPU-compute adapters are local mechanisms.

Figure 7 reports MECC at 7.38 s/token, streaming at 13.04, and fixed CPU at 21.75: TPOT ratios of 1.77× and 2.95×. The RF adapter takes 7.19 s, MECC I/O 7.78, and CPU compute 8.26; the RF adapter is the fastest local variant. MECC's TTFT is 139.1 s, versus 156.9 for streaming and 174.2 for fixed CPU. MECC, streaming, fixed CPU, RF, and MECC I/O report PPL 3.427; CPU compute reports 3.432. The

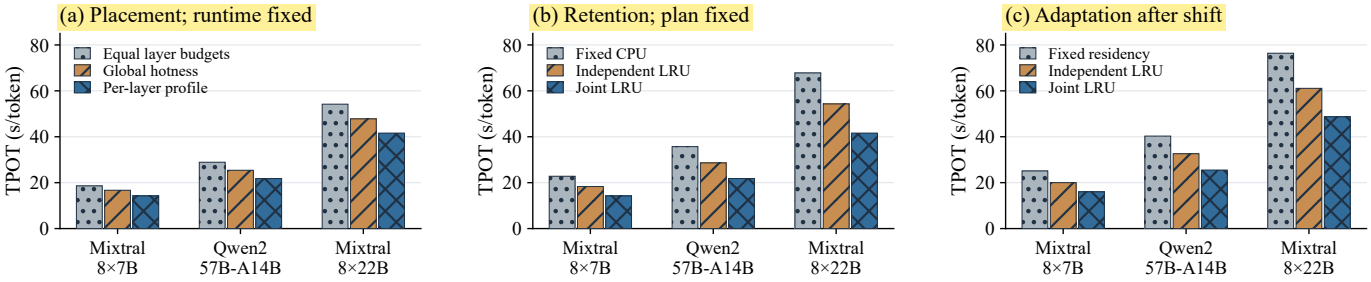


Fig. 8. Component comparisons on three models. (a) Placement with runtime fixed. (b) Retention with initialization fixed. (c) Adaptation after a task switch. The controls isolate initialization, victim retention, and online adaptation, respectively.

short-window route/logit/token diagnostic agrees between the local streaming and MECC paths.

C. Mechanism Attribution

The matched local independent-cache control takes 32.94 s/token. Figure 8 separates three mechanisms in the target component harness. For Mixtral-8x7B, the placement target moves from 18.6 to 14.2 s/token, joint retention from 18.3 to 14.2, and post-switch adaptation from 25.1 to 16.0 relative to fixed residency. The three controls hold runtime, initial plan, and switch protocol fixed, respectively. This separates better initial coverage from preservation of later reuse.

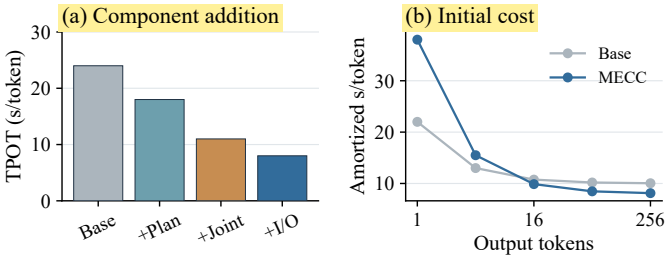


Fig. 9. Ablation and startup. The right panel reports initialization cost per output token plus decode cost.

Figure 9(a) compares component addition. Its TPOT sequence is 24, 18, 11, and 8 s/token. The last I/O step retains a 20-percent target HDD-demand share, separating delivery scheduling from miss reduction. Panel (b) uses $T_{\text{avg}}(n) = T_{\text{init}}/n + T_{\text{decode}}$ for n output tokens. The curves cross at 9 output tokens. Before that point, MECC's initial cost dominates. Later outputs amortize initialization and expose the difference in decode time.

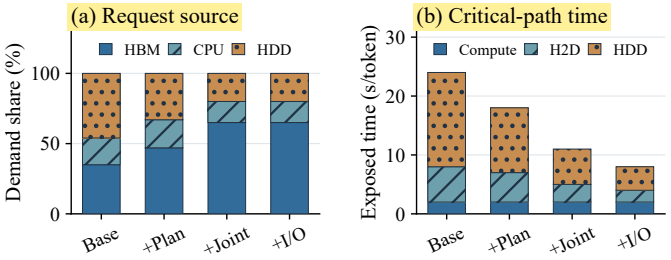


Fig. 10. Source demand and exposed time. The last I/O step preserves the demand fractions while shortening the exposed read/copy path.

Figure 10 separates cache coverage from scheduling. In the target decomposition, the HDD share falls from 46 to 20 percent before the I/O step. Overlap then shortens exposed delivery without changing the resident-set result. The stacked latency components sum to the TPOT targets in Figure 9. A source-demand share counts expert requests, while an exposed-time component counts delay on the execution path. Their changes need not be proportional: a host hit still requires H2D, and a cold read may overlap ready GPU work. Keeping both views distinguishes an improvement in cache coverage from an improvement in delivery timing.

D. Model and Deployment Scaling

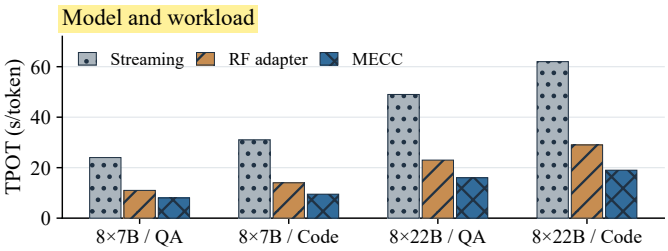


Fig. 11. Model and workload comparison. Mixtral-8x7B and 8x22B with QA/code inputs. Each policy uses the same model and workload within a group. The RF policy is a local adapter.

Figure 11 expands the present one-model workload. The MECC target ranges from 8 to 19 s/token, versus 24 to 62 for streaming and 11 to 29 for the RF adapter. QA and code exercise different expert-demand patterns; increasing model size also changes the expert pool and transfer size. The comparison therefore pairs every policy within each model/workload point.

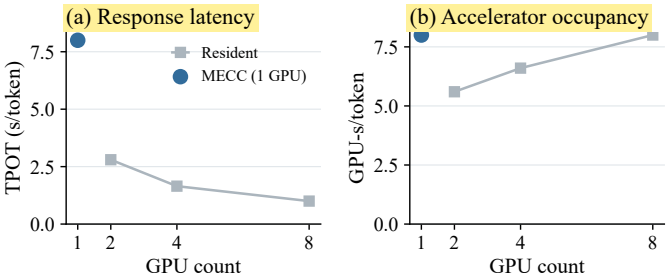


Fig. 12. One-GPU/resident-reference tradeoff. MECC has only a one-GPU point. Resident points use 2/4/8 GPUs; occupancy is GPU count times TPOT.

Figure 12 separates GPU provisioning from response latency. The endpoint target is one GPU at 8 s/token versus eight GPUs at 1 s/token; both correspond to 8 GPU-s/token. The two-GPU resident target is 5.6 GPU-s/token, which makes the smallest feasible resident configuration a necessary comparison. Occupancy is reported alongside latency, rather than substituted for energy.

E. Capacity and Workload Sensitivity

The measured 30/60 GB settings give MECC TPOTs of 14.46 and 0.76 s/token, with 32.2% and zero HDD demand. Four HBM plus

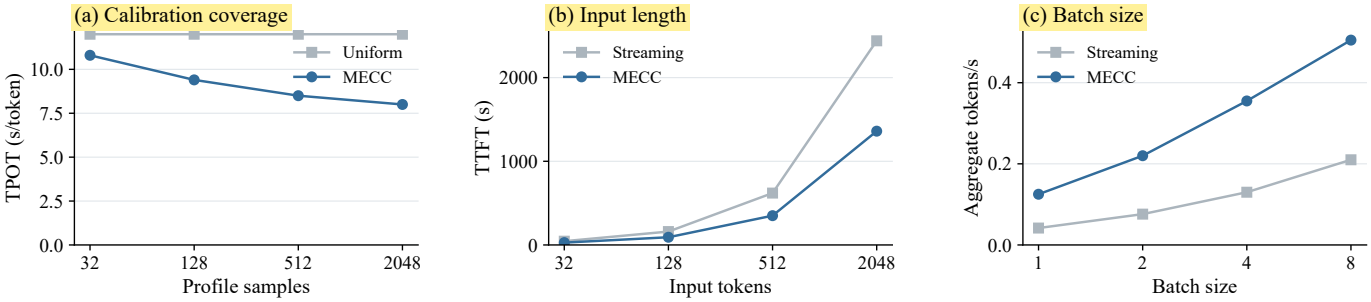


Fig. 13. Profile, input-length, and batch sensitivity. Profile size changes initialization quality, input length changes first-token work, and batch size changes aggregate throughput. The panels vary one workload parameter at a time.

four host slots at 60 GB cover all eight experts per layer; zero cold demand is a capacity transition. RF takes 13.06 and 0.74 s/token. These points use a shorter protocol than the main run.

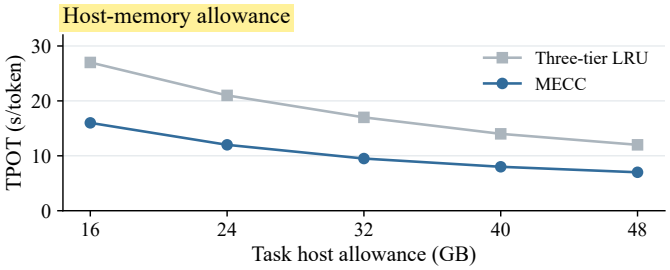


Fig. 14. Host-memory allowance sweep. Five host allowances compare MECC and three-tier LRU within one comparison protocol.

Figure 14 resolves intermediate capacity points. The MECC curve changes from 16 to 7 s/token over 16–48 GB, compared with 27 to 12 for three-tier LRU. The sweep separates pressure within the cold-tier regime from the full-coverage threshold. Additional host capacity can first preserve more reused victims, then eliminate cold demand once the complete expert set fits. The latter transition changes the dominant delivery path from HDD plus H2D to H2D alone. It explains why capacity and replacement policy must be examined together instead of extrapolating one speedup across all allowances.

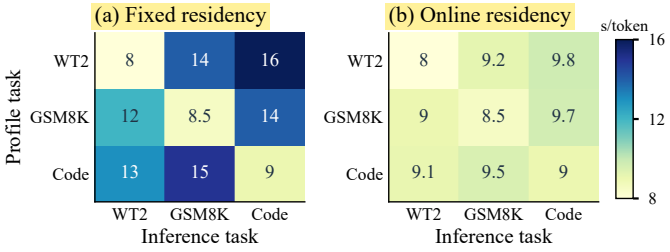


Fig. 15. Profile transfer across tasks. Rows select the calibration task and columns the inference task. Cells are TPOT in s/token; the two panels share a color scale.

Figure 13 examines scale within a workload. Figure 15 examines transfer between WikiText-2, GSM8K, and code inputs. Its target pattern distinguishes a useful initial prior from a permanently fixed placement: online membership updates reduce the off-diagonal penalty while retaining the matched-profile behavior.

VII. Related Work

Layer planning and online residency. MoE-CORE combines nonuniform device capacity, domain-informed initialization, routing-history-aware replacement, and cross-layer prefetching [13]. Its main path executes the router-selected experts; substitution is optional. MECC also separates initialization from online membership. Its focus is the connection between per-layer allocation and joint HBM/CPU retention when a selected expert can miss both tiers. Their overlap directs attention to the initial state, victim rule, and delivery path used by each comparison.

Storage-backed expert delivery. SSD-LLaMA manages bounded RAM and VRAM caches, retains GPU victims on the host when possible, and pipelines expert reads and transfers [14]. It also studies lossless transport compression and CPU/GPU execution partitioning. Edge FlashMoE studies device-cache replacement with on-demand storage access [15]. MECC keeps FP16 representation and GPU expert execution fixed in its retention controls. Fixing representation and execution placement isolates the residency decision within the shared delivery stack.

Offloading, prediction, and execution. Early expert offloading and SwapMoE expose placement and paging choices [8], [30]. Fiddler and DAOP choose CPU/GPU work; MoE-Lightning pipelines heterogeneous execution; MoE-Infinity uses activation traces [9]–[11], [31]. ProMoE, Pre-gated MoE, Fate, and ExpertFlow use proactive caching, lookahead, or token scheduling [12], [32]–[34]. HOBBIT and Specter vary loading precision; MoEpic uses segments; SliceMoE uses precision slices; Mira combines staging and quantization [35]–[39]. ZipMoE uses compression and affinity scheduling [40]. These mechanisms change demand prediction, bytes per request, or where computation runs. A comparison of whole-expert retention keeps those choices fixed.

Klotski and Bevy expose work across batches or requests [41], [42]. OD-MoE distributes on-demand loading across edge nodes; HeteGen studies heterogeneous dense inference [43], [44]. Their scheduling scopes differ from delivery within the current MoE gate. Ready work from another request can hide a wait even when the ongoing request has no resident expert to execute. This distinction separates throughput from the response time of one request.

VIII. Conclusion

MECC uses an offline plan to initialize a per-layer HBM/CPU expert cache. Online promotion preserves GPU victims on the host as routing changes. The implementation separates replacement from storage lifetime and orders reads and copies before execution. The A100 study reports decode latency, source-tier demand, and sample perplexity under a shared FP16 harness. Placement, retention, and delivery controls expose the decisions behind those outcomes.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[10] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[11] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[12] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[13] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[14] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[15] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[16] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[17] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[18] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[19] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[20] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[21] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[22] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[23] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[24] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[25] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[26] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[27] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[28] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[29] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[30] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[31] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[32] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[33] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[34] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[35] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[36] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[37] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[38] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[39] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[40] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[41] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[42] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[43] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

[44] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.